Otimização de hiperparâmetros de CNNs para Neural Ringer: propostas e ideias

1 Contexto e formulação

Este documento reúne cinco ideias de projeto que ligam a otimização de hiperparâmetros (HPO) de redes para o Neural Ringer em FPGA aos tópicos da disciplina de Otimização.

1.1 Variáveis de decisão

O vetor de hiperparâmetros θ reúne quatro grupos:

- **Arquitetura** θ_{arq}: número de camadas convolucionais, número de filtros F_ℓ e tamanho do kernel K_ℓ de cada camada ℓ, pooling e número de neurônios densos.
- **Treino** θ_{tr}: taxa de aprendizado, tamanho do lote, regularização. Não afetam o hardware.
- **Quantização** b: bits dos pesos b_i^w e das ativações b_i^a de cada camada i.
- **Paralelismo** RF_i: reuse factor de cada camada no hls4ml. Não altera a rede treinada, só o mapeamento no FPGA.

1.2 Função de desempenho: índice SP

O Ringer mede o desempenho pelo índice soma-produto (SP), que combina a probabilidade de detecção (PD) de elétrons e a taxa de falso alarme (FA) de jatos [1]:

$$\text{SP} = \sqrt{\sqrt{\text{PD}\,(1-\text{FA})}\cdot\frac{\text{PD}+(1-\text{FA})}{2}}, \qquad \text{PD} = \frac{TP}{TP+FN}, \qquad \text{FA} = \frac{FP}{FP+TN}. \tag{1}$$

TP são elétrons classificados como elétrons, FN são elétrons rejeitados, FP são jatos aceitos como elétrons e TN são jatos rejeitados. Como o treino é estocástico, a função objetivo usada na busca deve ser a média entre os K folds:

$$\overline{\text{SP}}(\theta) = \frac{1}{K}\sum_{k=1}^{K}\text{SP}_k(\theta). \tag{2}$$

1.3 Estrutura em dois níveis

Toda HPO é um problema em dois níveis. O nível interno é o treino: dados os hiperparâmetros θ, os pesos são

$$W^*(\theta) = \arg\min_W \; L_{\text{BCE}}(W;\theta). \tag{3}$$

O nível externo escolhe θ olhando o desempenho de validação de $W^*(\theta)$. Por isso a função $\overline{\text{SP}}(\theta)$ é cara (cada avaliação é um treino completo em K folds), ruidosa (depende da inicialização) e não tem gradiente em relação a θ. As propostas contornam isso de formas diferentes: busca sem derivadas (Proposta 1), modelos analíticos aproximados (Propostas 2 e 3), incorporação dos bits ao próprio treino (Proposta 4) e derivação implícita (Proposta 5).

1.4 Modelos de custo de hardware

Durante o treino não há síntese, então o custo de hardware entra por estimativas calculadas a partir da arquitetura:

- **MACs** de uma camada Conv1D e de uma camada densa:

$$\mathrm{MAC}_\ell^{\mathrm{conv}} = L_\ell^{\mathrm{out}}\, K_\ell\, C_\ell^{\mathrm{in}}\, C_\ell^{\mathrm{out}}, \qquad \mathrm{MAC}^{\mathrm{dense}} = n_{\mathrm{in}}\, n_{\mathrm{out}}, \tag{4}$$

  em que L^{out} é o comprimento da saída e C^{in}, C^{out} são os canais de entrada e saída.

- **BOPs** (operações de bit). A forma simples usada no HAWQ-V3 [8] é $G_i = b_i^w\, b_i^a\, \mathrm{MAC}_i$. O NAC [3] usa a forma completa

$$\mathrm{BOPs}^{\mathrm{dense}} = m\, n\, (p\, b^a b^w + b^a + b^w + \log_2 n)\,, \tag{5}$$

  para uma camada densa com n entradas, m saídas e fração p de pesos não podados; na convolução, n é multiplicado por k^2.

- **DSP**. No hls4ml, o consumo de DSP cai aproximadamente como 1/RF [5]:

$$\mathrm{DSP}_i \approx \left\lceil \frac{\mathrm{MAC}_i}{\mathrm{RF}_i} \right\rceil. \tag{6}$$

  Isso só vale para multiplicações largas: abaixo de cerca de 10 bits, o Vivado implementa as multiplicações em LUT em vez de DSP [5, 7].

- **LUT** cresce com o número de multiplicações e com os bits (aproximadamente com $\sum_i \mathrm{MAC}_i\, b_i^w\, b_i^a$ quando as multiplicações vão para LUT) e é pouco afetado pelo reuse factor [5].

- **Latência** cresce aproximadamente de forma linear com o reuse factor [5]. O consumo de BRAM não depende do reuse factor nos casos estudados em [5].

Esses modelos servem para guiar a busca e precisam ser calibrados com a síntese HLS (relatório do csynth) de algumas redes. Eles valem para o hls4ml. Como a implementação de [1] foi escrita em HDL, se o grupo mantiver a abordagem RTL será preciso um modelo de custo próprio, contando multiplicadores, somadores e registradores do circuito.

1.5 Problema mestre

Com $R_k(\theta)$ o consumo estimado do recurso k e B_k o orçamento disponível, o problema geral é:

$$\min_\theta \; -\overline{\mathrm{SP}}(\theta) \quad \text{sujeito a} \quad g_k(\theta) = \frac{R_k(\theta)}{B_k} - 1 \le 0, \qquad k \in \{\mathrm{LUT}, \mathrm{DSP}, \mathrm{FF}, \mathrm{BRAM}, \mathrm{Lat}\}. \tag{7}$$

A normalização por B_k coloca todas as restrições na mesma escala: $g_k = 0{,}2$ significa estourar o orçamento do recurso k em 20%.

1.6 Termos comuns

Termo	Significado
HPO	Hyperparameter Optimization: escolha das configurações fixadas antes do treino (arquitetura, bits, taxa de aprendizado).
NAS	Neural Architecture Search: HPO voltada à arquitetura da rede.
SP, PD, FA	Índice soma-produto, probabilidade de detecção e taxa de falso alarme (Seção 1.3).
BCE	Binary Cross-Entropy, a função de perda do treino: $-[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$.
K-fold	Validação cruzada: os dados são divididos em K partes; treina-se em $K - 1$ e valida-se na restante, K vezes.
MAC	Multiply-Accumulate: uma multiplicação seguida de soma acumulada; mede o tamanho computacional da rede.
BOPs	Bit Operations: MACs ponderados pelos bits dos operandos; estima o custo de multiplicações de precisão reduzida.
DSP, LUT, FF, BRAM	Recursos do FPGA: blocos multiplicadores dedicados, tabelas de consulta (lógica), flip-flops (registradores) e blocos de memória RAM.
RF	Reuse Factor do hls4ml: quantas vezes cada multiplicador é reutilizado por inferência. RF maior significa menos DSP e mais latência.
II	Initiation Interval: número de ciclos entre o início de duas inferências consecutivas.
HLS, hls4ml	High-Level Synthesis (síntese de hardware a partir de C++); hls4ml é a biblioteca que converte redes Keras/PyTorch em projetos HLS.
PTQ, QAT	Post-Training Quantization (quantizar depois de treinar) e Quantization-Aware Training (treinar já simulando a quantização).
STE	Straight-Through Estimator: no passo de retropropagação, a derivada da função de quantização é tratada como identidade [1].
Q$m.n$	Formato de ponto fixo com m bits inteiros, n bits fracionários e 1 bit de sinal. Q7.15 tem 23 bits.
Fronteira de Pareto	Conjunto das soluções em que não é possível melhorar um objetivo (SP) sem piorar outro (recursos).

2 Proposta 1: busca black-box com penalidade (TPE) e fronteira de Pareto (NSGA-II)

Tópicos da ementa: métodos de penalidade e barreira; tipos de extremos e condições de existência; otimização restrita.

2.1 Ideia

Tratar $\overline{\text{SP}}(\theta)$ como uma função caixa-preta: avalia-se, mas não se deriva. Um algoritmo de busca propõe configurações θ, cada uma é treinada e avaliada, e as restrições de hardware entram como penalidade na função objetivo, como restrições explícitas ou como um segundo objetivo.

2.2 O que os artigos fizeram

- **Aad et al. [2]** (redes da LAr do ATLAS para FPGA Agilex 7, incluindo uma CNN) otimizaram os hiperparâmetros por otimização bayesiana com processo gaussiano (kernel Matérn 5/2) e Expected Improvement. A função objetivo é a resolução de energia penalizada pelo número de MACs M:

$$f(M,\sigma) = \begin{cases} \tilde{\sigma}, & M \le 500, \\ \tilde{\sigma} + 0{,}3\,(\tilde{M} - 0{,}3), & 500 < M \le 850, \\ \tilde{\sigma} + 0{,}3\,(\tilde{M} - 0{,}3) + e^{\tilde{M}-0{,}65} - 1, & M > 850, \end{cases} \tag{8}$$

  com $\tilde{\sigma} = (\sigma - 0{,}07\ \text{GeV})/1{,}13\ \text{GeV}$ e $\tilde{M} = (M - 200)/1000$. As constantes tornam a função contínua: em $M = 500$ tem-se $\tilde{M} = 0{,}3$ e, em $M = 850$, $\tilde{M} = 0{,}65$. A busca começou com 10 pontos aleatórios e fez 30 iterações (100 para as CNNs), com 5 treinos por configuração para lidar com a aleatoriedade.

- **Weitz et al. [3]** (NAC) fizeram uma busca global com NSGA-II no Optuna, com dois objetivos (desempenho e BOPs), seguida de uma busca local com QAT em 4, 8, 16 e 32 bits e poda iterativa de 20% dos parâmetros por etapa. A busca global do caso BraggNN avaliou 1000 arquiteturas em 96 horas numa GPU.

- **Coelho et al. [4]** (AutoQKeras) trataram bits e número de filtros como hiperparâmetros, com busca aleatória, Hyperband ou processo gaussiano. O custo entra pelo *forgiving factor*

$$FF = 1 + \Delta_{\text{acc}} \cdot \log_R\!\left(S \cdot \frac{C_{\text{ref}}}{C_{\text{trial}}} \right), \tag{9}$$

  que multiplica a métrica de desempenho: Δ_{acc} é a perda de acurácia tolerada, C_{ref} e C_{trial} são os custos (energia ou bits) do modelo de referência e do candidato, e R e S controlam quanto menor o candidato precisa ser. Para limitar o espaço de busca, os blocos podem ser quantizados um de cada vez, o que reduz N^B combinações para $N \times B$ tentativas.

- **Ghielmetti et al. [6]** usaram o AutoQKeras numa CNN maior, aceitando até 4% de perda de acurácia se o consumo de recursos caísse pela metade.

- **Aarrestad et al. [5]** escolheram a arquitetura da CNN por otimização bayesiana (Keras Tuner) antes da compressão.

2.3 Formulação para o Ringer

(1a) Penalidade exterior quadrática. A restrição vira um termo que só atua quando é violada:

$$\min_{\theta}\ \Phi_\mu(\theta) = -\overline{\text{SP}}(\theta) + \mu \sum_k \big[\max(0,\, g_k(\theta))\big]^2. \tag{10}$$

$\mu > 0$ é o parâmetro de penalidade. Com μ pequeno a busca aceita violar o orçamento; com μ grande a função fica com um degrau íngreme na fronteira viável.

(1b) Penalidade por partes, adaptada de [2]. Com $\tilde{M} = \text{MAC}(\theta)/B_{\text{MAC}}$ (ou $\text{BOPs}/B_{\text{BOPs}}$):

$$f(\theta) = (1 - \overline{\text{SP}}(\theta)) + P(\tilde{M}), \qquad P(\tilde{M}) = \begin{cases} 0, & \tilde{M} \le 1, \\ \alpha(\tilde{M} - 1), & 1 < \tilde{M} \le \tilde{M}_{\max}, \\ \alpha(\tilde{M}_{\max} - 1) + e^{\gamma(\tilde{M} - \tilde{M}_{\max})} - 1, & \tilde{M} > \tilde{M}_{\max}. \end{cases} \tag{11}$$

α define o preço de cada unidade de excesso, $\tilde{M}_{\max}$ é o limite a partir do qual a rede não cabe mais no FPGA e γ define a rapidez do crescimento exponencial. A função é contínua em $\tilde{M} = 1$ e em $\tilde{M} = \tilde{M}_{\max}$.

(1c) Restrições explícitas. Em vez de penalizar, o amostrador recebe os $g_k(\theta)$ e prioriza configurações viáveis (no Optuna, argumento `constraints_func` dos amostradores TPE e NSGA-II). Não há parâmetro μ para ajustar.

(1d) Problema biobjetivo.

$$\min_{\theta} \; F(\theta) = \big(-\overline{\mathrm{SP}}(\theta),\; \mathrm{BOPs}(\theta) \big). \tag{12}$$

Uma configuração θ_1 domina θ_2 se não é pior em nenhum objetivo e é melhor em pelo menos um. A fronteira de Pareto é o conjunto das configurações não dominadas. O NSGA-II devolve uma aproximação dessa fronteira, e a escolha final é feita depois, de acordo com o orçamento do FPGA.

2.4 Ligação com a teoria da disciplina

- **Existência do mínimo.** Se θ toma valores num conjunto discreto e finito (filtros de 2 a 16, kernel de 2 a 7, e assim por diante), o mínimo existe, porque há finitas opções. Numa relaxação contínua, a existência vem do teorema de Weierstrass, que exige função contínua num conjunto compacto (limitado e fechado). Discutir qual hipótese vale em cada caso é um bom exercício.
- **Convergência da penalidade exterior.** Resolvendo uma sequência de problemas com $\mu_1 < \mu_2 < \cdots \to \infty$, os pontos de acumulação das soluções são soluções do problema restrito, sob hipóteses de continuidade. O preço é o mau condicionamento para μ grande, que no caso black-box aparece como um degrau difícil para o modelo substituto do TPE ou do processo gaussiano.
- **Por que não usar barreira aqui.** A barreira logarítmica

  $$\mathcal{B}(\theta) = -\sum_{k} \log\big(-g_k(\theta) \big) \tag{13}$$

  só existe no interior viável e vale $+\infty$ fora dele. Um amostrador black-box propõe pontos inviáveis, então a barreira não se aplica diretamente. Ela volta na Proposta 4, em que há gradiente e é possível partir de um ponto viável.
- **Limite da soma ponderada.** Minimizar $-\overline{\mathrm{SP}} + w \cdot \mathrm{BOPs}$ para vários pesos w só encontra pontos na parte convexa da fronteira de Pareto. O NSGA-II não tem essa limitação. Comparar as duas fronteiras mostra o efeito na prática.

2.5 Experimento proposto

- Espaço de busca para uma CNN-1D sobre os 50 anéis: 1 a 3 camadas convolucionais, 2 a 16 filtros, kernel de 2 a 7, 4 a 32 neurônios densos, taxa de aprendizado em escala logarítmica e, opcionalmente, bits de 4 a 16.
- Começar por uma região representativa (por exemplo $E_T \in [30, 40)$ GeV e $|\eta| \in [0;\, 0{,}8)$, a região usada nas curvas de treino de [1]) e depois estender às demais.
- Comparar, com o mesmo número de treinos (100 a 200): busca aleatória, TPE com penalidade (1a) para alguns valores de μ, TPE com penalidade por partes (1b), TPE com restrições (1c), NSGA-II (1d) e processo gaussiano com Expected Improvement, como em [2].
- Métricas: melhor $\overline{\mathrm{SP}}$ viável, número de treinos até a primeira solução viável, hipervolume da fronteira de Pareto e violação real dos recursos depois da síntese HLS das soluções finais.
- Usar o pruner do Optuna para interromper cedo treinos com desempenho ruim nas primeiras épocas e reduzir o custo.

2.6 Termos desta proposta

Termo	Significado
Busca black-box	Otimização que usa apenas valores da função, sem gradiente.
TPE	Tree-structured Parzen Estimator [13]: modela separadamente a densidade das boas e das más configurações e propõe pontos onde a razão entre elas é alta. É o amostrador padrão do Optuna.
NSGA-II	Non-dominated Sorting Genetic Algorithm II [14]: algoritmo genético multiobjetivo que ordena a população por dominância e mantém diversidade ao longo da fronteira.
Optuna	Biblioteca Python de HPO [12], com amostradores TPE, NSGA-II e processo gaussiano, além de pruners.
Otimização bayesiana, GP	Ajusta um modelo probabilístico (processo gaussiano, GP) aos pontos já avaliados e escolhe o próximo ponto por uma função de aquisição.
Kernel Matérn 5/2	Função de covariância do GP que supõe uma função objetivo duas vezes diferenciável, menos suave que o kernel gaussiano.
EI	Expected Improvement: valor esperado da melhora sobre o melhor ponto atual; equilibra exploração e aproveitamento.
Pruner, Hyperband	Interrupção antecipada de treinos ruins; o Hyperband distribui épocas entre muitas configurações e mantém só as melhores.
μ, g_k	Parâmetro de penalidade e restrição normalizada do recurso k ($g_k \leq 0$ significa dentro do orçamento).
Hipervolume	Área (ou volume) do espaço de objetivos dominada pela fronteira até um ponto de referência; mede a qualidade da fronteira.
Trial	Uma avaliação: um conjunto de hiperparâmetros treinado e validado.

3 Proposta 2: alocação de bits por Lagrange e KKT

Tópicos da ementa: multiplicadores de Lagrange; condições KKT; convexidade e condições de suficiência; método de Newton (Hessiana).

3.1 Ideia

Escolher quantos bits cada camada recebe (precisão mista) sob um orçamento de BOPs. Com um modelo aproximado do erro de quantização, o problema é convexo e tem solução em forma fechada pelas condições KKT. Essa solução pode ser comparada com a solução inteira exata da Proposta 3 e com os bits aprendidos da Proposta 4.

3.2 O que os artigos fizeram

- **HAWQ-V3 [8]** supõe que a perturbação causada pela quantização é independente entre camadas, $\Omega = \sum_i \Omega_i^{(b_i)}$, mede a sensibilidade de cada camada pela Hessiana e escolhe os bits por programação linear inteira com restrições de tamanho do modelo, BOPs e latência. O solver (PuLP) encontra a solução em menos de 1 segundo.
- **Campos et al. [9]** levaram esse fluxo ao hls4ml num classificador de jatos do trigger do LHC, com

$$\Omega_i = \mathrm{Tr}(H_i)\,\|Q(W_i) - W_i\|_2^2, \tag{14}$$

em que $\mathrm{Tr}(H_i)$ é o traço médio da Hessiana da camada i, estimado pelo algoritmo de Hutchinson,

$$\mathrm{Tr}(H) \approx \frac{1}{k} \sum_{j=1}^{k} z_j^\top H z_j, \tag{15}$$

com z_j de componentes ± 1 (distribuição de Rademacher). Resolveram o problema para limites de 250 mil a 400 mil BOPs. A solução escolhida (4, 4, 5 e 4 bits nas quatro camadas) usou 42,2% menos LUTs, 36,3% menos FFs e 95,7% menos DSPs que o modelo de referência.

- **Lisboa et al. [1]** usam a mesma precisão (Q7.15) em todas as camadas, sem perda de desempenho, o que indica margem para reduzir bits de forma diferenciada por camada.

3.3 Modelo da perturbação

Um quantizador uniforme com b bits sobre o intervalo $[-r_i, r_i)$ tem passo $\Delta_i = 2r_i/2^{b_i}$. Se o erro de arredondamento for uniforme, sua variância é $\Delta_i^2/12$, e para os n_i pesos da camada:

$$\|Q(W_i) - W_i\|_2^2 \approx \frac{n_i\, \Delta_i^2}{12} = \frac{n_i r_i^2}{3}\, 4^{-b_i}. \tag{16}$$

Substituindo na perturbação de [9]:

$$\Omega_i(b_i) \approx c_i\, 4^{-b_i}, \qquad c_i = \mathrm{Tr}(H_i)\, \frac{n_i r_i^2}{3}. \tag{17}$$

A justificativa vem da expansão de Taylor de segunda ordem da perda em torno dos pesos treinados: com gradiente aproximadamente nulo no mínimo, o aumento da perda é cerca de $\frac{1}{2}\, \Delta W^\top H\, \Delta W$, e a Hessiana mede o quanto cada camada é sensível a uma perturbação.

3.4 Problema

Fixando os bits das ativações, os BOPs ficam lineares nos bits dos pesos: $\mathrm{BOPs} = \sum_i a_i b_i$, com $a_i = \mathrm{MAC}_i\, b_i^a$. O problema é:

$$\min_b\ \Omega(b) = \sum_i c_i\, 4^{-b_i} \quad \text{sujeito a} \quad \sum_i a_i b_i \le B, \qquad b_{\min} \le b_i \le b_{\max}. \tag{18}$$

B é o orçamento de BOPs. Por enquanto os b_i são tratados como contínuos; o arredondamento vem depois (Seção 3.7).

3.5 Convexidade e condições KKT

Convexidade.

$$\frac{d^2}{db^2}\left(c_i 4^{-b}\right) = c_i (\ln 4)^2\, 4^{-b} > 0, \tag{19}$$

então a função objetivo é estritamente convexa (soma de funções convexas de uma variável). As restrições são lineares, e o conjunto viável é um poliedro. Se existe um ponto estritamente viável (condição de Slater, que vale quando $B > \sum_i a_i b_{\min}$), as condições KKT são necessárias e suficientes: todo ponto que as satisfaz é o mínimo global, e ele é único.

Lagrangiano. Com $\lambda \ge 0$ para o orçamento e $\nu_i^-, \nu_i^+ \ge 0$ para os limites:

$$\mathcal{L}(b, \lambda, \nu) = \sum_i c_i 4^{-b_i} + \lambda\Big(\sum_i a_i b_i - B\Big) + \sum_i \nu_i^-(b_{\min} - b_i) + \sum_i \nu_i^+(b_i - b_{\max}). \tag{20}$$

Condições KKT.

$$\begin{aligned}
&\text{Estacionariedade:} && -c_i \ln 4 \cdot 4^{-b_i} + \lambda a_i - \nu_i^- + \nu_i^+ = 0 \quad \forall i, && (21)\\
&\text{Viabilidade primal:} && \textstyle\sum_i a_i b_i \le B, \quad b_{\min} \le b_i \le b_{\max}, && (22)\\
&\text{Viabilidade dual:} && \lambda \ge 0, \quad \nu_i^- \ge 0, \quad \nu_i^+ \ge 0, && (23)\\
&\text{Folga complementar:} && \lambda\Big(\textstyle\sum_i a_i b_i - B\Big) = 0, \quad \nu_i^-(b_{\min} - b_i) = 0, \quad \nu_i^+(b_i - b_{\max}) = 0. && (24)
\end{aligned}$$

3.6 Solução

Para uma camada que não está em nenhum limite ($\nu_i^{\pm} = 0$), a estacionariedade dá:

$$b_i^*(\lambda) = \frac{1}{2}\log_2\!\left(\frac{c_i \ln 4}{\lambda\, a_i}\right). \tag{25}$$

Se nenhuma camada atinge os limites e o orçamento está ativo, substituindo em $\sum_i a_i b_i = B$:

$$\log_2 \lambda = \frac{\sum_i a_i \log_2(c_i \ln 4 / a_i) - 2B}{\sum_i a_i}. \tag{26}$$

Com limites, a solução é

$$b_i^*(\lambda) = \min\Big(b_{\max},\ \max\Big(b_{\min},\ \tfrac{1}{2}\log_2 \tfrac{c_i \ln 4}{\lambda a_i}\Big)\Big). \tag{27}$$

A função $h(\lambda) = \sum_i a_i b_i^*(\lambda) - B$ é não crescente em λ, então λ pode ser encontrado por bisseção. Se $\sum_i a_i b_{\max} \le B$, o orçamento não é ativo, $\lambda = 0$ e todas as camadas ficam em $b_{\max}$. Esse procedimento é conhecido como *water-filling*.

Interpretação. Camadas mais sensíveis (c_i maior) recebem mais bits, e camadas com mais MACs (a_i maior) recebem menos, porque cada bit delas custa mais BOPs. O multiplicador é o preço-sombra do orçamento:

$$\lambda = -\frac{d\Omega^*}{dB}, \tag{28}$$

a redução da perturbação ótima por unidade de BOPs a mais.

3.7 Onde entra o método de Newton

- **Hessiana sem formar a matriz.** O traço $\mathrm{Tr}(H_i)$ é estimado com produtos Hessiana-vetor (HVP), calculados por diferenciação automática com o custo de cerca de dois gradientes, como no algoritmo de Hutchinson [9, 17].
- **Newton no problema dual.** Em vez de bisseção, a equação $h(\lambda) = 0$ pode ser resolvida pelo método de Newton na variável $t = \log_2 \lambda$. Comparar o número de iterações e a robustez dos dois métodos (Newton converge mais rápido perto da raiz, mas pode falhar quando uma camada muda de limite) é um exercício direto da disciplina.

Arredondamento. Os b_i^* saem contínuos. Arredondá-los pode violar o orçamento ou perder otimalidade; a comparação com a solução inteira exata da Proposta 3 mede essa diferença.

3.8 Experimento proposto

- Aplicar primeiro à MLP atual do Ringer (duas camadas), como aquecimento, e depois à CNN.

- Treinar em ponto flutuante, estimar $\mathrm{Tr}(H_i)$ e r_i por camada, resolver (18) para vários orçamentos B, arredondar os bits e fazer um ajuste fino com QAT.
- Comparar o $\overline{\mathrm{SP}}$ com quantização uniforme de mesmo custo em BOPs e com a solução inteira da Proposta 3.
- Verificar se o modelo $\Omega_i \approx c_i 4^{-b_i}$ prevê a queda real de SP: medir a perda de validação para alguns valores de b_i e comparar com a previsão.

3.9 Termos desta proposta

Termo	**Significado**
Precisão mista	Cada camada (ou grupo de pesos) com seu próprio número de bits.
Ω_i, c_i, a_i	Perturbação da camada i, coeficiente de sensibilidade e custo em BOPs por bit de peso.
Hessiana H, traço $\mathrm{Tr}(H)$	Matriz das derivadas segundas da perda em relação aos pesos; o traço é a soma da diagonal e mede a curvatura média.
Hutchinson, Rademacher	Estimador estocástico do traço: média de $z^\top Hz$ para vetores z com componentes ± 1 sorteadas com probabilidade $\frac{1}{2}$.
HVP	Produto Hessiana-vetor Hz, calculado sem montar H.
λ, $\nu_i^{\pm}$	Multiplicadores de Lagrange do orçamento e dos limites de bits.
Condição de Slater	Existência de um ponto estritamente viável; em problemas convexos garante que KKT seja necessária.
Folga complementar	Um multiplicador só é positivo se a restrição correspondente estiver ativa (satisfeita com igualdade).
Preço-sombra	Variação do valor ótimo por unidade de folga na restrição; é o próprio multiplicador.
Water-filling	Solução em que um único nível (fixado por λ) define todas as variáveis, recortadas nos limites.

4 Proposta 3: programação linear inteira e SIMPLEX

Tópicos da ementa: programação linear; método SIMPLEX; relaxação linear e branch-and-bound.

4.1 Ideia

Quando cada camada tem um conjunto pequeno de opções (bits e reuse factor), a escolha ótima é um problema de programação linear inteira (ILP). O mesmo formato serve para decidir quais regiões do comitê recebem redes maiores quando todas dividem o mesmo FPGA.

4.2 O que os artigos fizeram

- **HAWQ-V3** [8] formula:

$$\begin{aligned} \min_{\{b_i\}} \quad & \sum_{i=1}^{L} \Omega_i^{(b_i)} \\ \text{sujeito a} \quad & \sum_i M_i^{(b_i)} \le \text{limite de tamanho}, \\ & \sum_i G_i^{(b_i)} \le \text{limite de BOPs}, \\ & \sum_i Q_i^{(b_i)} \le \text{limite de latência}, \end{aligned} \tag{29}$$

  em que M_i, G_i e Q_i são o tamanho, os BOPs e a latência da camada i com b_i bits. Com L camadas e B opções por camada, há B^L combinações.

- **Campos et al.** [9] usaram essa formulação com restrição de BOPs e levaram a solução ao hls4ml (resultados na Seção 3.2).

- **Ramhorst et al.** [7] formularam a poda como um problema da mochila 0-1: os pesos processados pelo mesmo DSP (definidos pelo reuse factor) formam grupos w_i, e escolhe-se o subconjunto $\hat{W}$ que resolve

$$\max_{\hat{W}} \sum_{w_i \in \hat{W}} \frac{\|w_i\|_1}{\max_{w_j \in L} \|w_j\|_1} \quad \text{sujeito a} \quad \sum_{w_i \in \hat{W}} R(w_i) \preceq c, \tag{30}$$

  em que $R(w_i)$ é o consumo (DSP, BRAM) do grupo e c é o orçamento. Na forma padrão, $\max_x v^\top x$ sujeito a $Ux \preceq c$, $x \in \{0,1\}^n$, resolvido por branch-and-cut. A redução de DSP ficou entre 55% e 92% e a de BRAM chegou a 81%.

4.3 Formulação A: configuração por camada

Para cada camada i, enumera-se um conjunto J_i de opções j (par de bits e reuse factor). A variável binária x_{ij} vale 1 se a camada i usa a opção j. Para cada opção, mede-se ou estima-se a perda de qualidade Δ_{ij} (por exemplo Ω_i da Proposta 2 ou a queda de SP medida) e o custo de cada recurso.

$$\begin{aligned} \min_{x} \quad & \sum_i \sum_{j \in J_i} \Delta_{ij}\, x_{ij} \\ \text{sujeito a} \quad & \sum_{j \in J_i} x_{ij} = 1 \quad \forall i, \\ & \sum_{i,j} \mathrm{DSP}_{ij}\, x_{ij} \le B_{\mathrm{DSP}}, \qquad \sum_{i,j} \mathrm{LUT}_{ij}\, x_{ij} \le B_{\mathrm{LUT}}, \qquad \sum_{i,j} \mathrm{Lat}_{ij}\, x_{ij} \le T, \\ & x_{ij} \in \{0,1\}. \end{aligned} \tag{31}$$

A primeira restrição obriga exatamente uma opção por camada. Os custos vêm do modelo da Seção 1.5: $\mathrm{DSP}_{ij} \approx \lceil \mathrm{MAC}_i / \mathrm{RF}_j \rceil$ quando os bits passam de cerca de 10 (e perto de zero abaixo disso), LUT crescendo com os bits e latência crescendo com o RF. Somar as latências supõe camadas executadas em sequência, como na implementação `io_parallel`.

4.4 Formulação B: comitê de 25 regiões

Para cada região r (5 faixas de E_T × 5 de η), toma-se um conjunto de configurações candidatas j, por exemplo pontos da fronteira de Pareto da Proposta 1, com desempenho SP_{rj} e consumo

R^k_{rj}. Com $y_{rj} \in \{0,1\}$ e w_r o peso da região (por exemplo a fração de eventos):

$$\max_{y} \sum_{r} \sum_{j} w_r \, \mathrm{SP}_{rj} \, y_{rj} \quad \text{sujeito a} \quad \sum_{j} y_{rj} = 1 \;\; \forall r, \qquad \sum_{r,j} R^k_{rj} \, y_{rj} \le B_k \;\; \forall k. \tag{32}$$

Esta é uma mochila de múltipla escolha e supõe redes diferentes instanciadas em paralelo, cada uma com seu próprio caminho de dados. A arquitetura de [1] é diferente: os 25 especialistas compartilham um único caminho de dados, e só os parâmetros são trocados. Nesse caso todas as regiões usam a mesma arquitetura e a mesma precisão, e o problema se reduz a escolher uma configuração comum j,

$$\max_{j} \sum_{r} w_r \, \mathrm{SP}_{rj} \quad \text{sujeito a} \quad R^k_j \le B_k \;\; \forall k, \tag{33}$$

que se resolve por enumeração. A formulação B faz sentido se o grupo avaliar caminhos de dados separados, por exemplo um por grupo de faixas de η; decidir quantos caminhos usar e quais regiões vão para cada um é uma extensão possível, também como ILP.

4.5 Ligação com o SIMPLEX

- **Relaxação linear.** Trocando $x_{ij} \in \{0,1\}$ por $0 \le x_{ij} \le 1$, o problema vira uma PL resolvida pelo SIMPLEX. O valor ótimo da relaxação é um limite inferior (na minimização) para o valor inteiro.
- **Poucas variáveis fracionárias.** Numa solução básica, o número de variáveis básicas é igual ao número de restrições: G restrições de escolha (uma por camada ou região) mais m restrições de recursos. Como cada grupo precisa de pelo menos uma variável positiva, no máximo m grupos ficam com valores fracionários. Com um ou dois recursos, a relaxação já é quase inteira, e o arredondamento costuma ser bom. Demonstrar isso é um exercício da disciplina.
- **Dualidade.** As variáveis duais das restrições de recursos são os preços-sombra de DSP, LUT e latência, análogos ao λ da Proposta 2.
- **Branch-and-bound.** O solver inteiro resolve uma relaxação linear em cada nó da árvore e descarta ramos cujo limite já é pior que a melhor solução inteira conhecida. Branch-and-cut, usado em [7], acrescenta planos de corte a esse processo.

4.6 Experimento proposto

- Construir as tabelas Δ_{ij} e de custos para a CNN escolhida na Proposta 1 e resolver (31) com `scipy.optimize.milp` (solver HiGHS) ou PuLP, como em [8].
- Comparar três soluções: relaxação linear arredondada, solução inteira exata e solução contínua da Proposta 2 arredondada. Medir a distância de otimalidade e a violação de recursos.
- Sintetizar as soluções no hls4ml e comparar DSP, LUT e latência medidos com os previstos.
- Se houver interesse em caminhos de dados separados, montar (32) com candidatos de algumas regiões.

4.7 Termos desta proposta

Termo	Significado
PL, ILP	Programação linear; programação linear inteira (Integer Linear Programming), em que algumas variáveis devem ser inteiras.
Variável binária x_{ij}, y_{rj}	Indica se a opção j foi escolhida para a camada i ou para a região r.
Relaxação linear	O mesmo problema com as variáveis inteiras tratadas como contínuas; fornece um limite para o ótimo inteiro.
SIMPLEX	Método que percorre vértices (soluções básicas) do poliedro viável até o ótimo de uma PL.
Solução básica	Vértice do poliedro, definido por um conjunto de variáveis básicas igual ao número de restrições.
Branch-and-bound, branch-and-cut	Métodos exatos para ILP que dividem o problema em subproblemas e usam relaxações lineares (e cortes) como limites.
Mochila 0-1, mochila de múltipla escolha	Escolher itens de valor máximo sem exceder a capacidade; na versão de múltipla escolha, exatamente um item por grupo.
PuLP, HiGHS	Modelador de PL em Python [8] e solver de PL/ILP de código aberto usado pelo SciPy.

5 Proposta 4: bits aprendidos por gradiente (HGQ) com subida dual

Tópicos da ementa: método do gradiente; projeção de gradiente; multiplicadores de Lagrange e dualidade; métodos de barreira.

5.1 Ideia

Em vez de escolher os bits por busca ou por um modelo analítico, os bits passam a ser parâmetros treinados junto com os pesos, e o custo de hardware entra na função de perda. O peso dado ao custo pode ser fixo (penalidade) ou ajustado durante o treino para cumprir um orçamento (subida dual).

5.2 O que os artigos fizeram

- **Lisboa et al. [1]** usaram QAT com STE: no passo direto, pesos e entradas são quantizados; no passo de retropropagação, a derivada da quantização é tratada como identidade, e as atualizações são aplicadas a uma cópia dos pesos em ponto flutuante. Eles observaram que o QAT exige mais épocas que o modelo em ponto flutuante, o que atribuem ao ruído introduzido pelo STE.
- **Sun et al. [10]** (HGQ, citado em [1]) tornam treinável o número de bits de cada peso ou grupo de pesos, com um gradiente substituto para a largura de bits, e acrescentam à perda um termo $\beta \cdot \text{EBOPs}$, uma estimativa do custo em hardware calculada a partir dos bits efetivamente usados. Um peso que chega a zero bits é removido, então a poda acontece no mesmo processo. Esse artigo não está entre os anexos; a descrição aqui segue a versão publicada.
- **Ramhorst et al. [7]** também incluem uma regularização ligada aos recursos durante o treino, que empurra para zero os grupos de pesos que compartilham o mesmo DSP.

5.3 Formulação

Versão penalizada. Com W os pesos e f os bits (fracionários) treináveis:

$$\min_{W,f} \; L(W,f) = L_{\text{BCE}}(W,f) + \beta \cdot \text{EBOPs}(f). \tag{34}$$

Cada valor de β gera uma rede diferente; variando β, obtém-se pontos da fronteira desempenho $\times$ custo. Pela mesma razão da Seção 2.4, a soma ponderada só alcança a parte convexa da fronteira.

Versão restrita. Para cumprir um orçamento B de EBOPs:

$$\min_{W,f} \; L_{\text{BCE}}(W,f) \quad \text{sujeito a} \quad \text{EBOPs}(f) \le B, \qquad f_{\min} \le f \le f_{\max}, \tag{35}$$

com Lagrangiano

$$\mathcal{L}(W,f,\beta) = L_{\text{BCE}}(W,f) + \beta(\text{EBOPs}(f) - B), \qquad \beta \ge 0. \tag{36}$$

Aqui β passa a ser o multiplicador de Lagrange do orçamento.

5.4 Algoritmo primal-dual com projeção

A cada passo (ou época) de treino:

1. Passo de gradiente nos pesos: $W \leftarrow W - \eta\, \nabla_W \mathcal{L}$.
2. Passo de gradiente projetado nos bits: $f \leftarrow \Pi_{[f_{\min},\, f_{\max}]}\big(f - \eta\, \nabla_f \mathcal{L}\big)$, com
$$\Pi_C(x) = \arg\min_{y \in C} \|y - x\|, \tag{37}$$
que num intervalo é o recorte (*clip*) de cada componente.
3. Subida dual no multiplicador: $\beta \leftarrow \max\big(0,\; \beta + \eta_\beta\, (\text{EBOPs}(f) - B)\big)$.

Se o custo está acima do orçamento, β aumenta e a rede é empurrada para menos bits; se sobra folga, β diminui até zero, o que reproduz a folga complementar. O tamanho de passo η_β controla a estabilidade: valores grandes fazem β oscilar.

5.5 Alternativa com barreira

Como aqui há gradiente, a barreira logarítmica se aplica:

$$\min_{W,f} \; L_{\text{BCE}}(W,f) - \rho \log\big(B - \text{EBOPs}(f)\big), \qquad \rho \to 0. \tag{38}$$

O termo cresce sem limite quando EBOPs se aproxima de B, então o treino nunca sai da região viável, desde que comece dentro dela (por exemplo, inicializando com poucos bits). ρ é reduzido ao longo do treino. A comparação com a penalidade fixa e com a subida dual (velocidade, estabilidade e violação final) é o núcleo do experimento.

5.6 Experimento proposto

- Treinar a CNN (ou a MLP atual) com HGQ2 para vários valores de β e traçar a fronteira $\overline{\text{SP}} \times$ EBOPs.
- Repetir com subida dual para alguns orçamentos B e com a barreira, e comparar o $\overline{\text{SP}}$ final, o número de épocas e se o orçamento foi cumprido.
- Comparar os bits aprendidos por camada com a alocação analítica da Proposta 2.
- Sintetizar os pontos finais e verificar a relação entre EBOPs e LUT/DSP reais.

5.7 Termos desta proposta

Termo	Significado
HGQ, HGQ2	High Granularity Quantization [10]: quantização com largura de bits treinável por peso ou grupo; HGQ2 é a versão para Keras 3.
EBOPs	Effective BOPs: estimativa de custo em hardware calculada com os bits efetivamente usados por cada multiplicação.
f	Bits fracionários treináveis; definem o passo de quantização de cada peso ou grupo.
Gradiente substituto	Derivada aproximada usada no lugar de uma operação não diferenciável (arredondamento, número de bits).
β	Peso do custo na perda; na versão restrita, multiplicador de Lagrange do orçamento.
Projeção Π_C	Ponto de C mais próximo de x; num intervalo, é o recorte nos limites.
Subida dual, primal-dual	Atualizar o multiplicador na direção da violação da restrição, alternando com passos de descida nas variáveis primais.
Barreira logarítmica, ρ	Termo $-\rho \log(\text{folga})$, que impede sair da região viável; ρ é reduzido ao longo das iterações.

6 Proposta 5 (opcional): hipergradiente com gradiente conjugado

Tópico da ementa: método de direções conjugadas.

6.1 Ideia

Para hiperparâmetros contínuos (decaimento de pesos, o β da Proposta 4, a intensidade de uma regularização L1), é possível calcular a derivada da perda de validação em relação ao hiperparâmetro e otimizá-lo por gradiente. O cálculo exige resolver um sistema linear com a Hessiana, e o gradiente conjugado resolve esse sistema usando só produtos Hessiana-vetor. As referências são Lorraine et al. [15] e Pedregosa [16], que não estão entre os anexos.

6.2 Formulação

Problema em dois níveis com hiperparâmetro λ:

$$\min_{\lambda} \; L_{\text{val}}\big(W^*(\lambda), \lambda\big) \quad \text{com} \quad W^*(\lambda) = \arg\min_{W} L_{\text{tr}}(W, \lambda). \tag{39}$$

No mínimo interno, $\nabla_W L_{\text{tr}}(W^*(\lambda), \lambda) = 0$. Derivando essa identidade em λ (teorema da função implícita), com $H = \nabla^2_W L_{\text{tr}}$:

$$\frac{dW^*}{d\lambda} = -H^{-1}\, \nabla^2_{W\lambda} L_{\text{tr}}, \qquad \frac{dL_{\text{val}}}{d\lambda} = \frac{\partial L_{\text{val}}}{\partial \lambda} - v^\top \nabla^2_{W\lambda} L_{\text{tr}}, \quad \text{em que} \quad Hv = \nabla_W L_{\text{val}}. \tag{40}$$

Exemplo: para decaimento de pesos, $L_{\text{tr}} = L_{\text{BCE}} + \frac{\lambda}{2}\|W\|^2$, tem-se $\nabla^2_{W\lambda} L_{\text{tr}} = W$, e o hipergradiente é $-v^\top W$.

A perda de validação precisa ser diferenciável, então usa-se a BCE de validação, não o SP, que depende de limiares de decisão.

6.3 Ligação com direções conjugadas

- Resolver $Hv = g$, com H simétrica definida positiva, equivale a minimizar a quadrática $\frac{1}{2}v^\top Hv - g^\top v$. O gradiente conjugado gera direções H-conjugadas ($p_i^\top Hp_j = 0$ para $i \neq j$) e, em aritmética exata, converge em no máximo n iterações.
- Cada iteração precisa apenas de um produto Hp, calculado por diferenciação automática, sem montar H.
- Em redes neurais, H pode não ser definida positiva. As correções usuais são o amortecimento ($H + \delta I$) ou a matriz de Gauss-Newton, que é semidefinida positiva. Além disso, W^* é só aproximado, porque o treino para antes do mínimo exato. Discutir essas limitações faz parte do trabalho.

6.4 Experimento proposto

Otimizar o decaimento de pesos (ou o β do HGQ) por hipergradiente, comparando o gradiente conjugado com a série de Neumann usada em [15] quanto ao número de produtos Hessiana-vetor e à precisão do hipergradiente, e comparando o resultado final com uma busca TPE pelo mesmo hiperparâmetro.

6.5 Termos desta proposta

Termo	**Significado**
Hipergradiente	Derivada da perda de validação em relação a um hiperparâmetro.
Teorema da função implícita	Permite derivar $W^*(\lambda)$ a partir da condição de otimalidade $\nabla_W L_{\text{tr}} = 0$, sem diferenciar o treino passo a passo.
Direções H-conjugadas	Direções p_i com $p_i^\top Hp_j = 0$; minimizar ao longo de cada uma não desfaz o progresso nas anteriores.
Amortecimento, Gauss-Newton	Correções que tornam a matriz definida positiva para que o gradiente conjugado funcione.
Série de Neumann	Aproximação $H^{-1} \approx \alpha \sum_j (I - \alpha H)^j$, usada em [15].

7 Ementa × propostas

Tópico da ementa	Proposta	Onde aparece
Tipos de extremos e condições de existência	1	Espaço discreto finito × relaxação contínua (Weierstrass)
Condições de otimalidade e critérios de extremo	2, 5	Estacionariedade do Lagrangiano; $\nabla_W L_{\text{tr}} = 0$ no nível interno
Condições de suficiência e convexidade	2	Convexidade estrita de $\sum_i c_i 4^{-b_i}$; KKT suficiente
Multiplicadores de Lagrange	2, 4	λ como preço-sombra dos BOPs; β como multiplicador do orçamento de EBOPs
Condições KKT	2	Solução water-filling com limites de bits e folga complementar
Método do gradiente	4	Treino dos pesos e dos bits
Método de Newton	2	Hessiana como sensibilidade; Newton na equação dual $h(\lambda) = 0$
Direções conjugadas	5	Solução de $Hv = \nabla L_{\text{val}}$ por gradiente conjugado
Projeção de gradiente	4	Recorte dos bits no intervalo permitido
Penalidade e barreira	1, 4	Penalidade exterior e por partes na busca; barreira logarítmica no treino
Programação linear e SIMPLEX	3	ILP de bits e reuse factor; relaxação linear; comitê

8 Ordem sugerida

1. Proposta 1 numa região do detector, para escolher a arquitetura da CNN com custo de hardware estimado.
2. Proposta 2 na MLP atual e depois na CNN escolhida, para alocar bits por camada.
3. Proposta 3 para obter a solução inteira exata e comparar com a Proposta 2; incluir o reuse factor quando a síntese entrar no estudo.
4. Proposta 4 como alternativa aprendida às Propostas 2 e 3.
5. Síntese HLS dos pontos finais para calibrar os modelos de custo da Seção 1.5.
6. Proposta 5, se houver tempo.

Referências

[1] G. H. B. Lisboa, L. B. A. Nunes, P. H. G. Lima *et al.* Redes Neurais Quantizadas para Filtragem Online de Elétrons em um Experimento de Altas Energias. *XLIV Simpósio Brasileiro de Telecomunicações e Processamento de Sinais (SBrT 2026)*, Salvador, 2026.

[2] G. Aad, R. Bertrand, L. Laatu, E. Monnier, A. Straessner, N. Sur, J. C. Voigt. Optimised neural networks for online processing of ATLAS calorimeter data on FPGAs. *Eur. Phys. J. C* 86:111, 2026. doi:10.1140/epjc/s10052-026-15321-y.

[3] J. Weitz, D. Demler, L. McDermott, N. Tran, J. Duarte. Neural architecture codesign for fast physics applications. *Mach. Learn.: Sci. Technol.* 6, 035009, 2025. doi:10.1088/2632-2153/adede1.

[4] C. N. Coelho Jr., A. Kuusela, S. Li, H. Zhuang, T. Aarrestad, V. Loncar, J. Ngadiuba, M. Pierini, A. A. Pol, S. Summers. Automatic heterogeneous quantization of deep neural networks for low-latency inference on the edge for particle detectors. *Nat. Mach. Intell.* 3, 675–686, 2021. arXiv:2006.10159.

[5] T. Aarrestad, V. Loncar, N. Ghielmetti, M. Pierini, S. Summers, J. Ngadiuba *et al.* Fast convolutional neural networks on FPGAs with hls4ml. *Mach. Learn.: Sci. Technol.* 2, 045015, 2021. arXiv:2101.05108.

[6] N. Ghielmetti, V. Loncar, M. Pierini, M. Roed, S. Summers, T. Aarrestad *et al.* Real-time semantic segmentation on FPGAs for autonomous vehicles with hls4ml. 2022. arXiv:2205.07690.

[7] B. Ramhorst, V. Lončar, G. A. Constantinides. FPGA Resource-aware Structured Pruning for Real-Time Neural Networks. 2023. arXiv:2308.05170.

[8] Z. Yao, Z. Dong, Z. Zheng, A. Gholami *et al.* HAWQ-V3: Dyadic Neural Network Quantization. *ICML*, 2021. arXiv:2011.10680.

[9] J. Campos, J. Mitrevski, N. Tran, Z. Dong, A. Gholaminejad, M. W. Mahoney, J. Duarte. End-to-end codesign of Hessian-aware quantized neural networks for FPGAs. *ACM Trans. Reconfigurable Technol. Syst.* 17(3), art. 36, 2024. doi:10.1145/3662000.

[10] C. Sun, Z. Que, T. Aarrestad, V. Loncar, J. Ngadiuba, W. Luk, M. Spiropulu. HGQ: High Granularity Quantization for Real-time Neural Networks on FPGAs. *FPGA '26*, p. 79–91, 2026. doi:10.1145/3748173.3779200.

[11] Z. Dong, Z. Yao, D. Arfeen, A. Gholami, M. W. Mahoney, K. Keutzer. HAWQ-V2: Hessian Aware trace-Weighted Quantization of Neural Networks. *NeurIPS*, 2020.

[12] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama. Optuna: A Next-generation Hyperparameter Optimization Framework. *KDD*, 2019. arXiv:1907.10902.

[13] J. Bergstra, R. Bardenet, Y. Bengio, B. Kégl. Algorithms for Hyper-Parameter Optimization. *NeurIPS*, 2011.

[14] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Trans. Evol. Comput.* 6(2), 182–197, 2002.

[15] J. Lorraine, P. Vicol, D. Duvenaud. Optimizing Millions of Hyperparameters by Implicit Differentiation. *AISTATS*, 2020. arXiv:1911.02590.

[16] F. Pedregosa. Hyperparameter optimization with approximate gradient. *ICML*, 2016. arXiv:1602.02355.

[17] M. F. Hutchinson. A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines. *Commun. Stat. Simul. Comput.* 19(2), 433–450, 1990.